

Builder Pattern: Email

Java implementation and design report

Student: Шалкар | Group: SE-2532 | Submission: 12 September 2026, 16:00

1. Introduction

This project demonstrates the Builder design pattern using an email as the product. The same construction steps create two representations: an immutable Email object for use in Java code and a readable String preview. Email is one of the permitted products and differs from the supplied Custom PC example.

The construction sequence is separated from the result representation. EmailDirector contains a welcome-email recipe and an order-confirmation recipe. It accepts only EmailBuilder, so each recipe can work with either concrete builder.

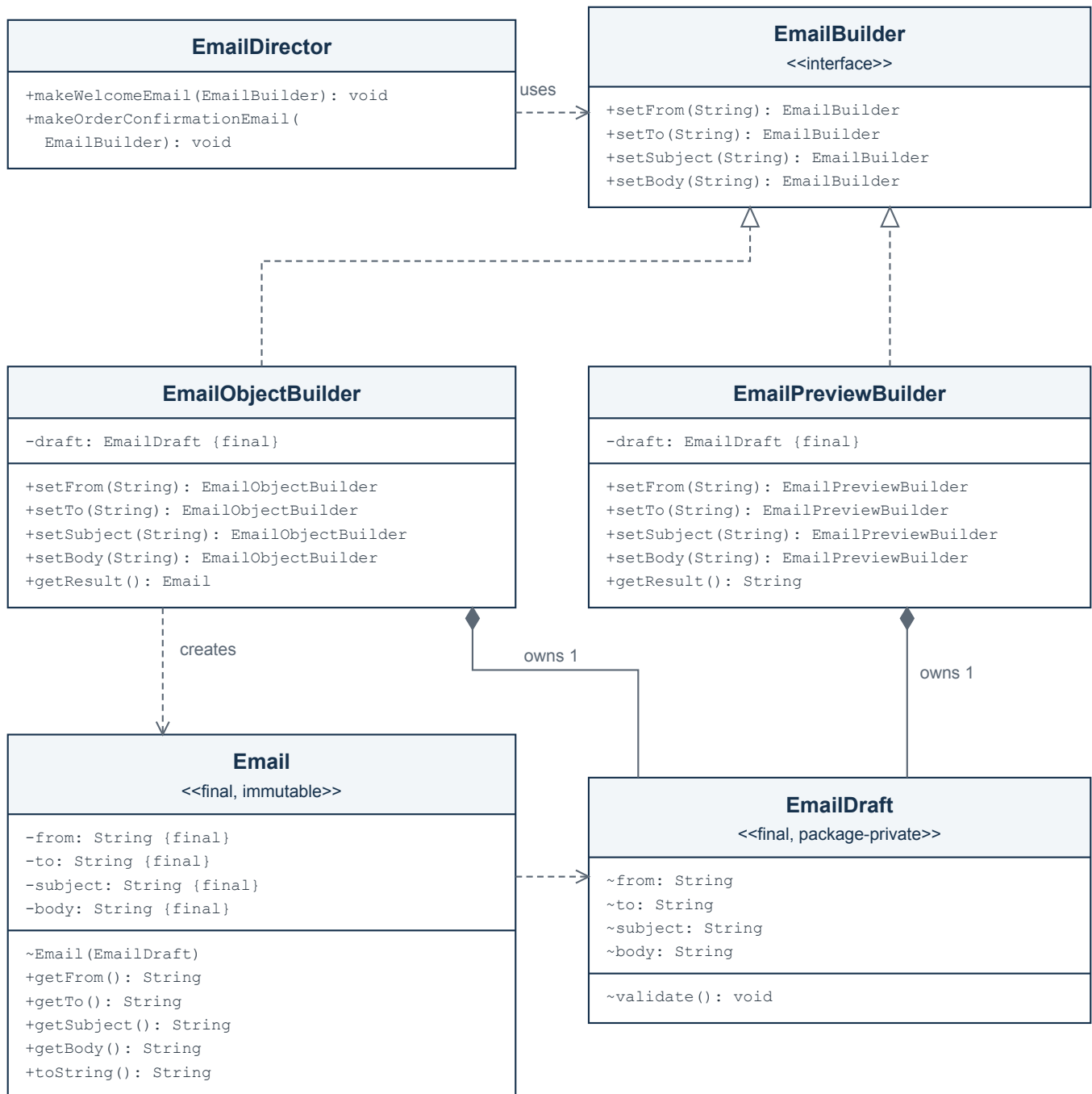
2. Design and construction

Pattern role	Implementation
Immutable product	Email: private final fields, no setters, package-private constructor.
Builder contract	EmailBuilder: setFrom, setTo, setSubject and setBody.
Object representation	EmailObjectBuilder.getResult() returns Email.
Text representation	EmailPreviewBuilder.getResult() returns String.
Director	EmailDirector: makeWelcomeEmail and makeOrderConfirmationEmail.
Internal state	Each builder owns a separate EmailDraft with shared validation rules.
Client	Main creates the builders, calls both recipes and prints both results.

Both recipes use this order: sender, recipient, subject, body. Every step returns the same builder. getResult() belongs to each concrete builder because Email and String have different types; the director only needs the common construction steps.

The application is a local console demonstration. It does not send email or require network access, a server, Maven, Gradle or external Java libraries.

3. UML class diagram



Legend: + public; - private; ~ package-private. Dashed lines with hollow triangles mean interface implementation; dashed arrows mean dependency; filled diamonds mean ownership. Each builder owns one draft. Email copies draft values during construction and does not retain the draft.

Main is the client and is omitted to keep the pattern relationships readable. Private recipe constants, the preview template and private validation helpers are also omitted. The text result is Java's immutable String. Editable source: docs/uml.puml.

4. Clean Code principles

All excerpts below are extracted from the submitted Java source files.

1. Meaningful names and named constants

Recipe values have names that explain their purpose. This avoids unexplained configuration literals in the construction steps and keeps changes in one place.

```
private static final String SUPPORT_ADDRESS = "support@example.com";
private static final String ORDERS_ADDRESS = "orders@example.com";
private static final String CUSTOMER_ADDRESS = "customer@example.com";
private static final String WELCOME_SUBJECT = "Welcome to Study Shop";
```

EmailDirector.java, lines 5-8

2. Single responsibility

The preview builder formats text; the object builder creates Email; the director defines recipes. EmailDraft owns the common validation rules. Changing preview formatting therefore does not require changing the object product or the director.

```
public String getResult() {
    draft.validate();
    return PREVIEW_TEMPLATE.formatted(
        draft.from, draft.to, draft.subject, draft.body);
}
```

EmailPreviewBuilder.java, lines 40-44

3. Depend on an abstraction

The director's parameter is EmailBuilder. It can apply the same recipe to either representation and never calls a concrete getResult(). A third implementation of the interface could reuse this recipe without editing the director.

```
public void makeWelcomeEmail(EmailBuilder builder) {
    builder.setFrom(SUPPORT_ADDRESS)
        .setTo(CUSTOMER_ADDRESS)
        .setSubject(WELCOME_SUBJECT)
        .setBody(WELCOME_BODY);
}
```

EmailDirector.java, lines 15-20

4. Clean Code principles

Shared rules, safe products and clear failures

4. Do not repeat validation rules

Both concrete `getResult()` methods call `draft.validate()`. A required-field or address rule is maintained in one helper instead of being duplicated and drifting between the object and text representations.

```
void validate() {
    requireAddress(from, "From");
    requireAddress(to, "To");
    requireText(subject, "Subject");
    requireSingleLine(subject, "Subject");
    requireText(body, "Body");
}
```

EmailDraft.java, lines 16-22

5. Encapsulation and immutability

Email is final, its state is private and final, and it exposes no setters. Its package-private constructor takes one internal draft and copies four immutable String values. The mutable draft itself is not stored in the product.

```
public final class Email {
    private final String from;
    private final String to;
    private final String subject;
    private final String body;

    Email(EmailDraft draft) {
        this.from = draft.from;
        this.to = draft.to;
        this.subject = draft.subject;
        this.body = draft.body;
    }
}
```

Email.java, lines 4-15

6. Clear errors at the result boundary

A partially filled draft is allowed during construction, but `getResult()` rejects invalid state. The exception identifies the missing field, making the failure understandable to the caller.

```
private static void requireText(String value, String field) {
    if (value == null || value.isBlank()) {
        throw new IllegalStateException(field + " is required");
    }
}
```

EmailDraft.java, lines 31-35

5. Validation and conclusion

7. Small methods and a fluent API

Each construction step performs one assignment and returns this. Concrete return types preserve chaining while implementing the EmailBuilder interface.

```
public EmailObjectBuilder setSubject(String subject) {
    draft.subject = subject;
    return this;
}
```

EmailObjectBuilder.java, lines 20-23

Validation and executed checks

All four fields are required and must contain non-whitespace text. Sender and recipient must match the documented simple email-address rule. Subjects must be single-line; message bodies may be multiline. Invalid state throws IllegalStateException from either getResult(). The address rule is intentionally limited and does not implement the full Internet email specification.

Check area	Observed result
Build and execution	Compiled using JDK 24.0.1 with --release 17, -Xlint:all and -Werror. Main ran successfully.
Both recipes	Object fields and complete preview text match each recipe.
Invalid input	Both builders reject null/blank fields, malformed addresses and multiline subjects.
Fluent and reuse behavior	Every step returns the same builder. Repeated steps replace values; existing results remain unchanged.
Test runner	./test.sh completed: PASS: 64 behavioral checks.

Builders retain their current draft and are intended for one thread. Both director recipes overwrite every field. Calling getResult() repeatedly does not append text or bypass validation. Editing a builder cannot change an earlier Email or String.

Conclusion

The implementation separates construction from representation using one fluent interface, two concrete builders and a director with two reusable recipes. The Email product is immutable, both builders validate their result, and the design avoids a long public constructor and repeated validation logic.

GitHub and submission

<https://github.com/Shalkar111/builder-email-assignment>

The repository contains the Java source, README and staged publication commits. The development-history.bundle file preserves the four original local development commits and their exact history.